

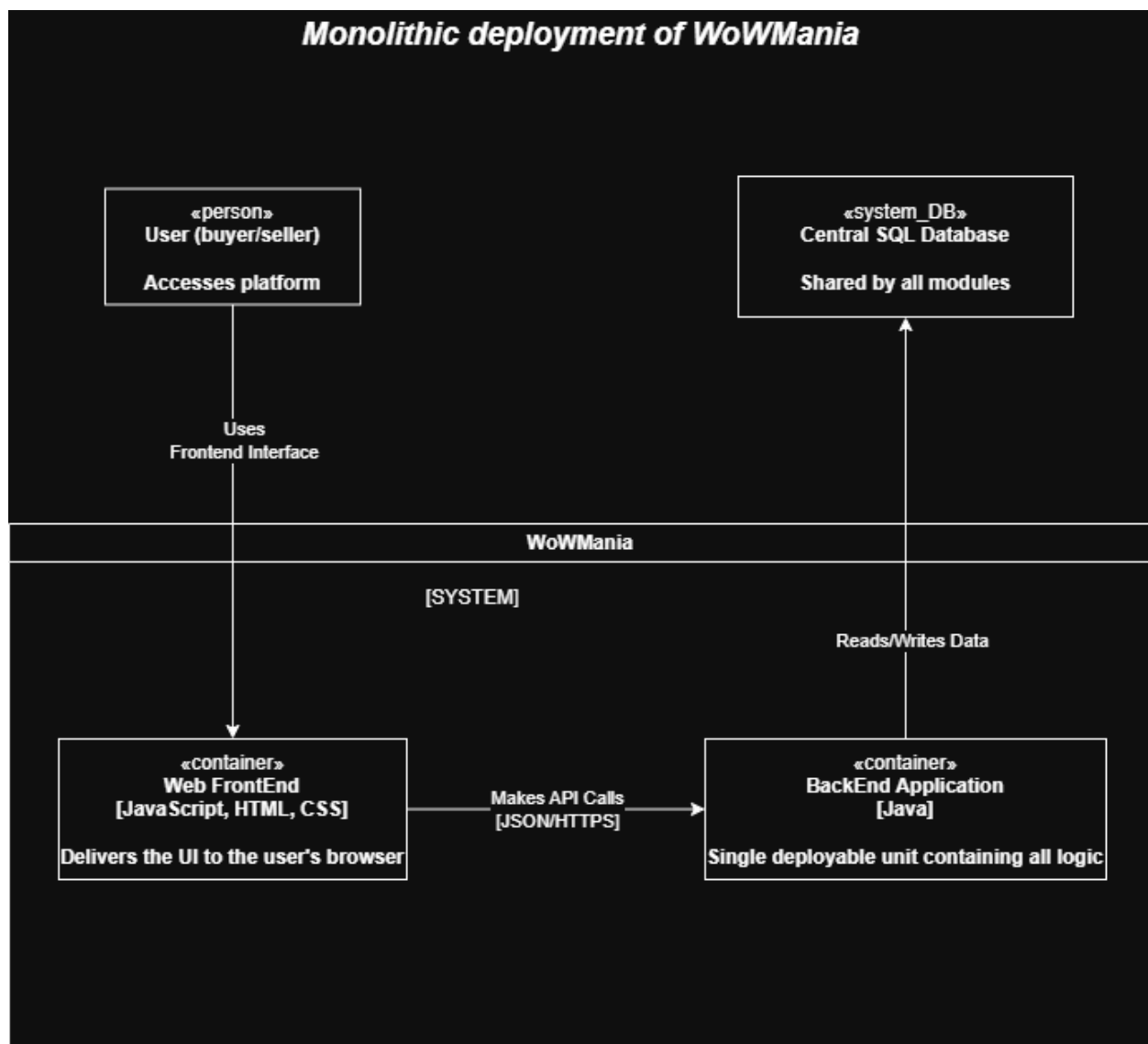
Munteanu Amalia-Nicole
Francu Teodor Matei
Radu Matei

WowMania Project: Milestone 3

1. Monolithic Architecture

If we were to use the Monolithic Architecture, we would build WowMania as a single, unified application. All the code for our features - User management, the Marketplace, the LFG system, and the Agreement tracking - would be in one large codebase and deployed as a single unit.

Structure and Data Flow: The components would just be modules inside one big application. When a user creates a listing, the ListingModule would call the UserModule directly as a simple, in-memory function call. All modules would share one single, central database, meaning tables for Users, Listings, and Agreements would all exist in the same schema.



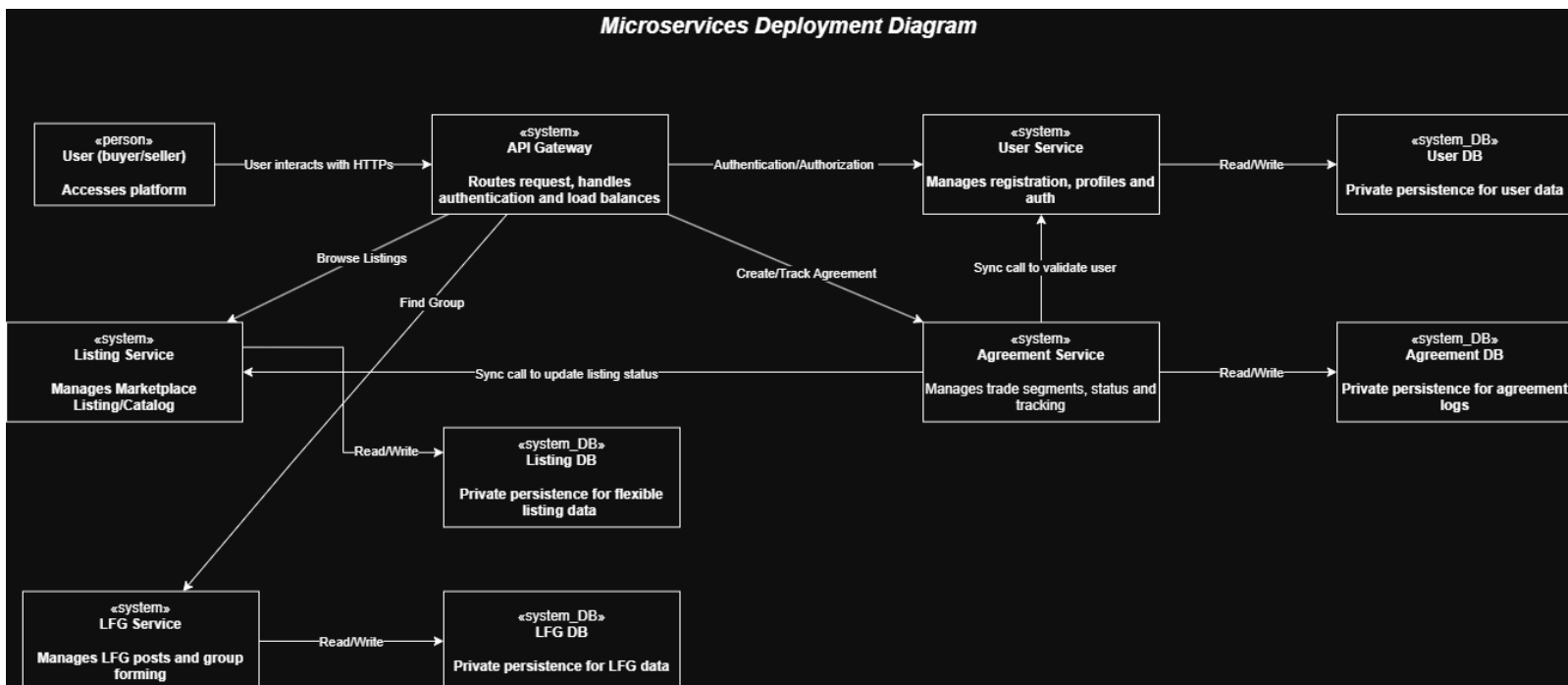
Pros and Cons: The main advantage is simplicity. It would be much faster for our team to develop, test, and debug this way at the start. Since our system doesn't involve real money, the fact that all data is in one database makes it very easy to keep our agreement logs and listing statuses consistent by using a single, simple ACID transaction.

The major disadvantage is the complete lack of scalability and fault tolerance. If the LFG system becomes popular and gets hit with thousands of users, the entire application slows down, which would make the Marketplace unusable. Furthermore, it has no fault isolation. A critical bug or memory leak in the LFG feature would crash the entire site, stopping all users from making or tracking agreements.

2. Microservices Architecture

On the other hand, the Microservices Architecture breaks WowMania apart into a set of small, independent services, with each service handling one specific business capability.

Structure and Data Flow: We would have a UserService, a ListingService, an AgreementService, and a separate LFGService. Each one would be its own mini-application. To make them work together, we'd use an API Gateway as the front door to route requests. Each service would have its own database.



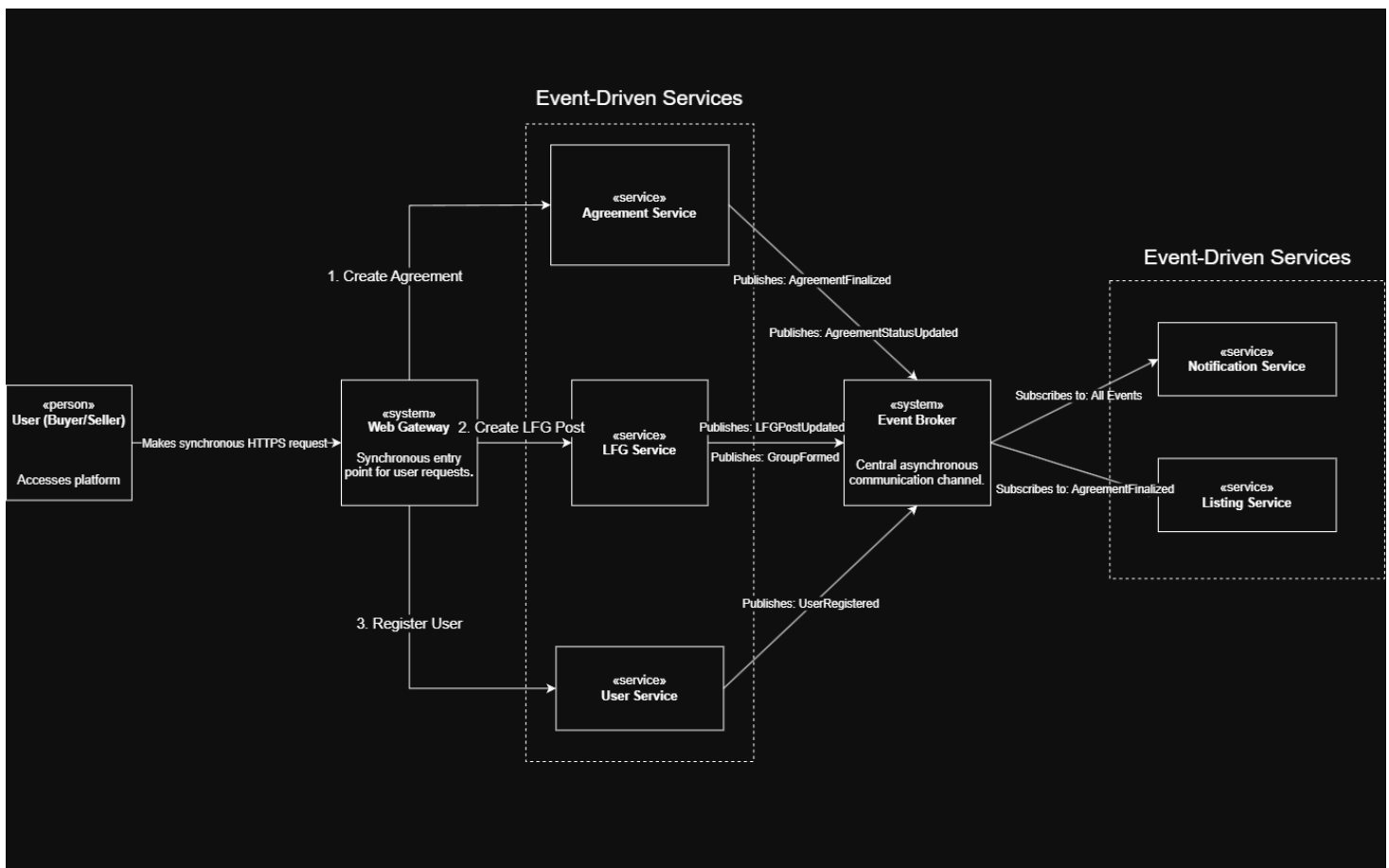
Pros and Cons: The main advantage is the fact that we would now have scalability. We can scale just the LFGService 10x to handle a traffic spike, without touching the stable AgreementService. This also gives us high fault isolation. If the LFGService crashes, the Marketplace and agreement system stay online.

The main cons are rooted in its distributed complexity. It introduces network latency for every service-to-service call and requires complex patterns like the Saga Pattern to keep data consistent across different databases (for example ensuring a listing is marked Sold after an agreement is made).

3. Event-Driven Architecture

Like the Microservices Architecture, the Event-Driven Architecture is also a distributed style, however, it's focused on asynchronous communication. Instead of services calling each other directly, they just publish "events" to a central message broker (like Kafka). Other services then "subscribe" to the events they care about.

Structure and Data Flow: In this architecture, the Event Broker is the center of this design. The AgreementService doesn't know or care about the ListingService. It just publishes an event, and its job is done. The ListingService (and our NotificationService for the Observer pattern) would both be subscribed to that event and would react on their own time.



Pros and Cons: It is the most flexible and scalable option. It would be a perfect fit for the Observer Pattern, as notifications are just another subscriber to the event stream. It's also extremely fault-tolerant. If the ListingService is down, the "AgreementFinalized" events just pile up in the broker, and the service can process them when it comes back online.

The major con is eventual consistency (this means it might take a few seconds for a listing to be marked Sold after an agreement is made). This architecture is also, by far, the most complex to debug. Trying to trace what happened from a single user click is very difficult when it's just a chain of events.

Conclusion:

After evaluating all three architecture types, we believe from the start that the Monolithic architecture is the wrong choice, since it fails the main requirement for scalability and fault tolerance (the LFG feature would break the code down). Thus, we would have to choose between Microservices and Event-Driven architectures. Even though the Event-Driven style is a powerful and perfect fit for our LFG system, we consider that it would be too complex for a small Marketplace. Thus, the Microservices Architecture is the most suitable choice for WowMania. It directly solves our main scaling problem by isolating the LFGService from the AgreementService. At the same time, it still allows us to keep our most critical business logic (validating and creating an agreement) synchronous and predictable.